

Pequeño repaso Concurrencia

Dr. Luis Gustavo Esquivel Quirós

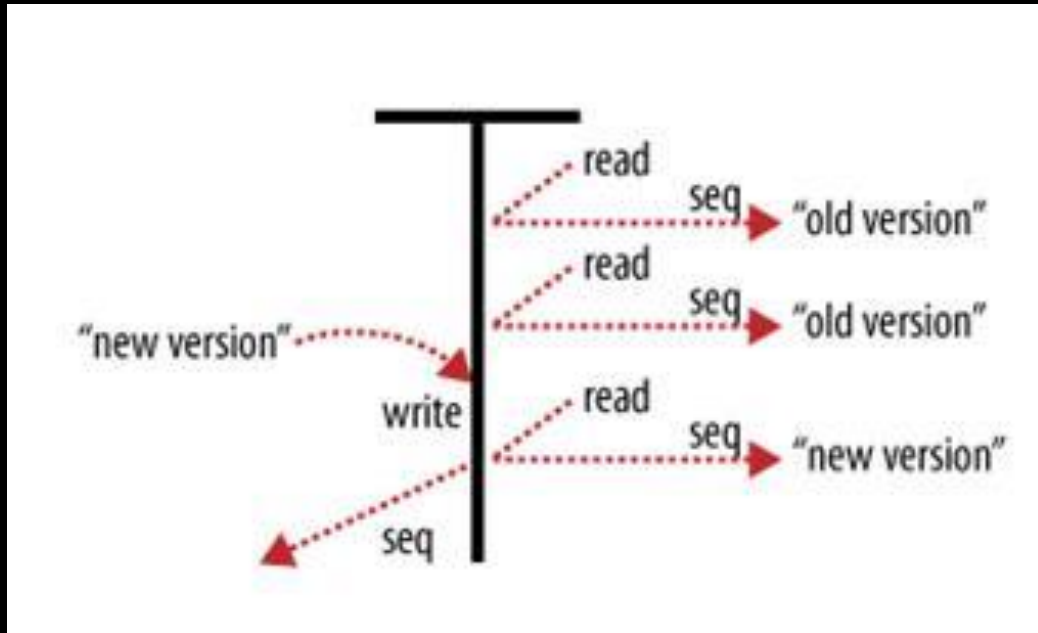




Un concepto importante: MVCC

- Multi-Version Concurrency Control (control de concurrencia multiversión)
- Cada actualización crea una nueva versión de los objetos de datos
- Por lo tanto, los lectores simultáneos pueden leer la versión antigua del dato => Las transacciones de solo lectura nunca tienen que esperar
- Pero la mayoría de las implementaciones solo brindan aislamiento de instantáneas

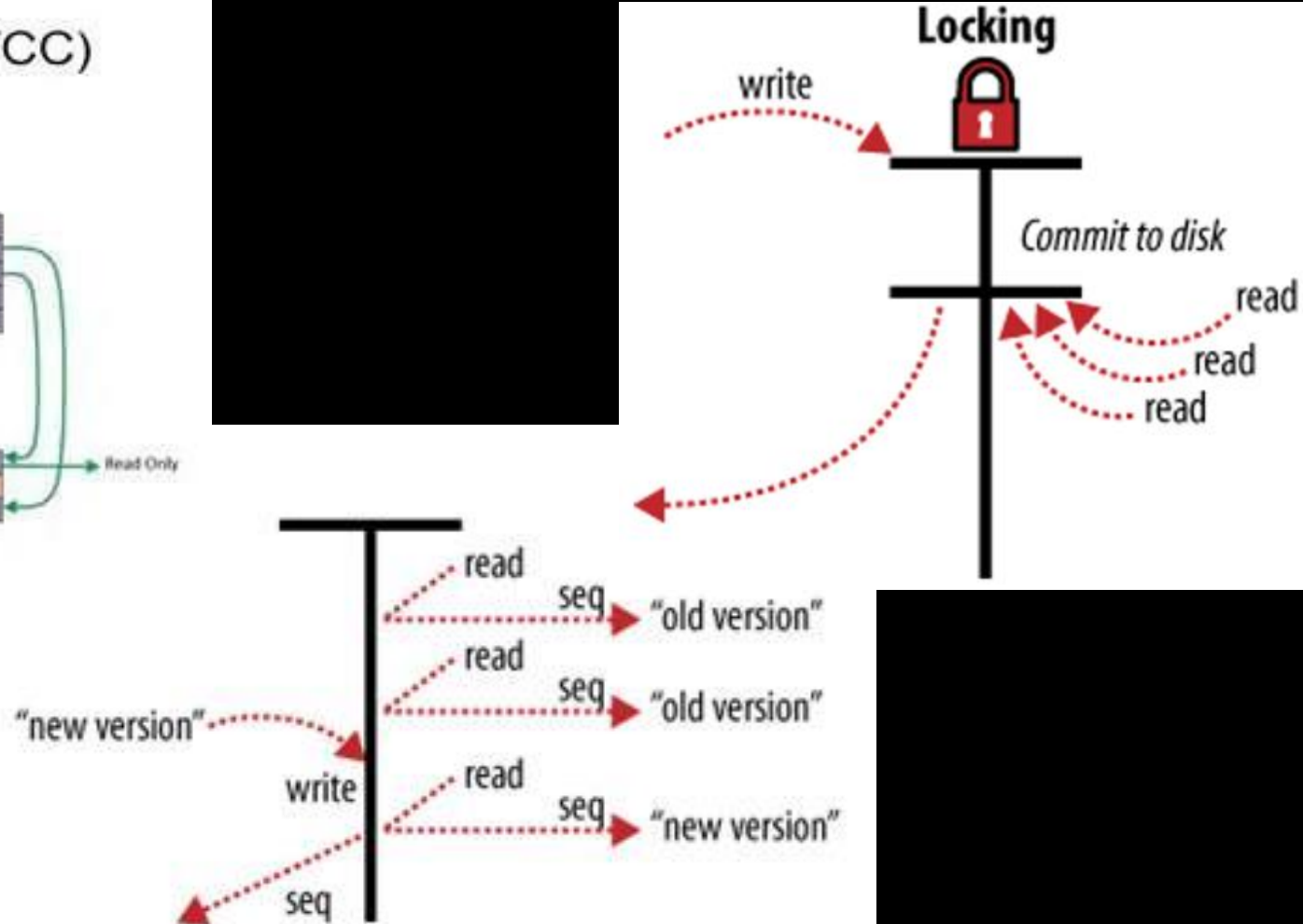
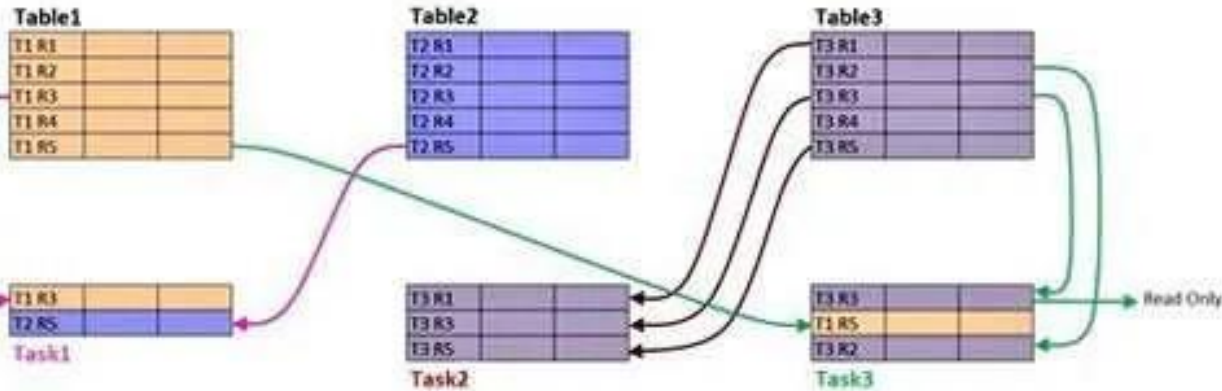
Control de concurrencia multiversión



- La base de datos mantiene múltiples versiones físicas de un registro
- Durante la escritura de datos en una transacción, se crea una nueva versión de los datos escritos
- Las solicitudes de lectura obtienen la última versión de los datos en función de la información de la instantánea creada al comienzo de la transacción o declaración
- Cuando se utiliza este método, la escritura no bloquea la lectura y la lectura no bloquea la escritura
- Las solicitudes de lectura nunca fallarán debido a tales conflictos (como en transacciones de versión única) ni esperarán (como en 2PL de versión única).

Control de concurrencia multiversión

Multi-Version Concurrency Control (MVCC)

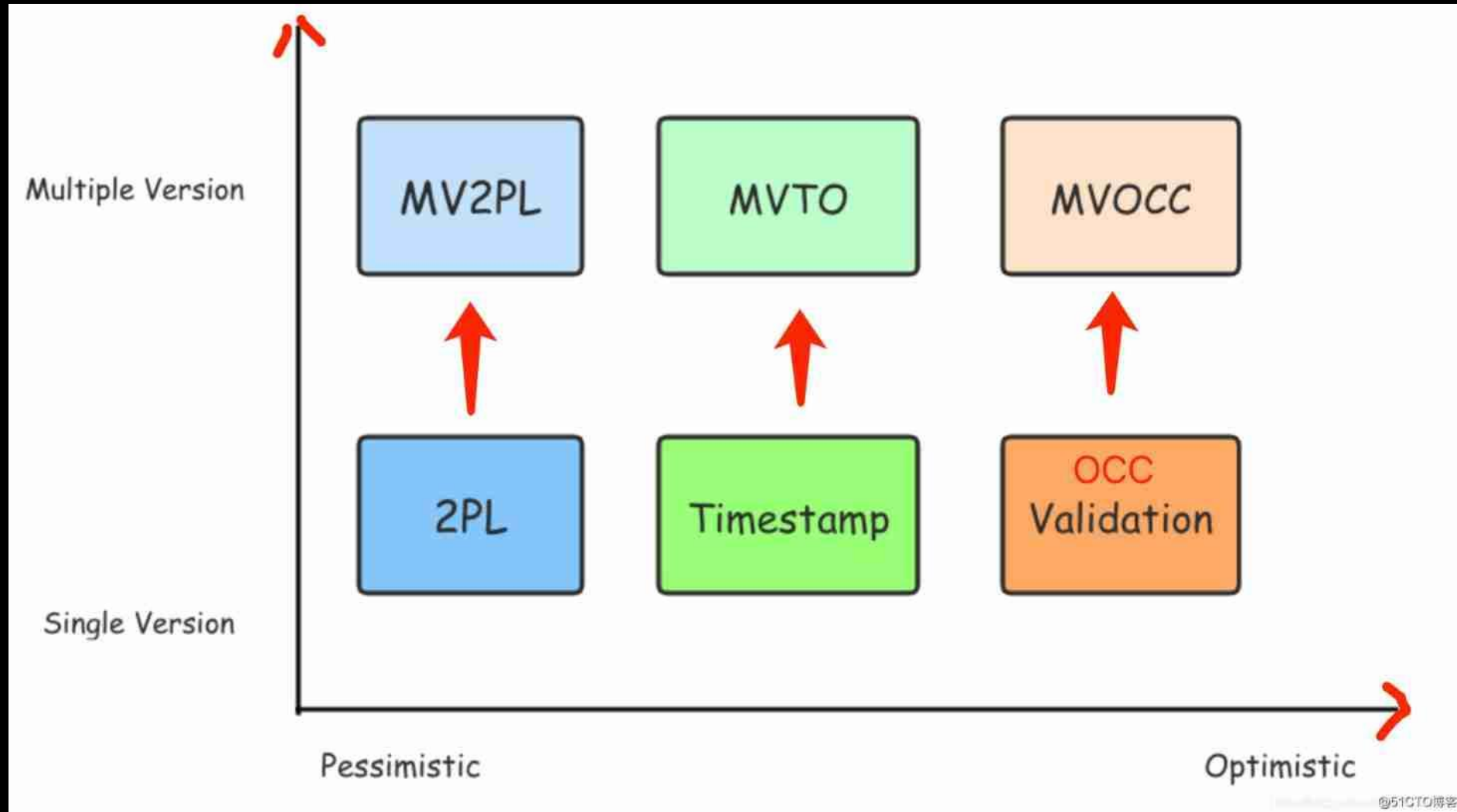


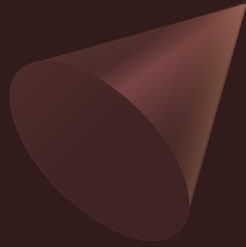
Control de concurrencia multiversión

- En las bases de datos, las solicitudes de lectura generalmente superan en número a las solicitudes de escritura. Por lo tanto, casi todas las bases de datos utilizan esta tecnología de optimización
- El control de concurrencia multiversión (MVCC) es una técnica de optimización para solicitudes de lectura y escritura.
- No resuelve completamente el problema de concurrencia de las bases de datos
- Debe usarse en conjunto con las técnicas de control de concurrencia:
 - El bloqueo de dos fases de múltiples versiones (MV2PL)
 - El orden de marca de tiempo de múltiples versiones (MVTO)
 - El control de concurrencia optimista de múltiples versiones (MVOCC).



Control de concurrencia multiversión





Propiedades ACID de las transacciones



Atomicidad: O todas las operaciones que forman parte de la transacción se completan exitosamente o ninguna lo hace.

Consistencia: cada transacción de forma aislada conserva un conjunto de restricciones de integridad de los datos.

Aislamiento: el comportamiento de las transacciones no se ve afectado (más de lo deseado) por presencia de otras transacciones concurrentes.

Durabilidad: los efectos de la transacción sobreviven a la falla del almacenamiento volátil (memoria) o no volátil (disco).

Contexto

Definiciones:

- Operación: normalmente se refiere a un acceso único operación (por ejemplo, leer, escribir)
- Transacción: una o más operaciones que deben comprometerse atómicamente



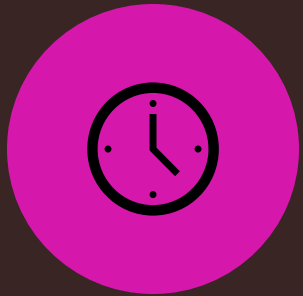
Linealización



Garantía para una sola operación en un solo objeto



De manera informal, las escrituras deberían aparecer instantáneamente dentro del sistema



Todas las lecturas posteriores definidas por el tiempo del reloj de pared (es decir, en tiempo real) reflejan el valor escrito o algún valor escrito posterior

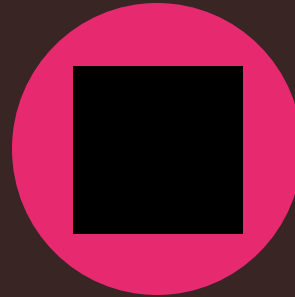


"Consistencia fuerte" en el teorema CAP

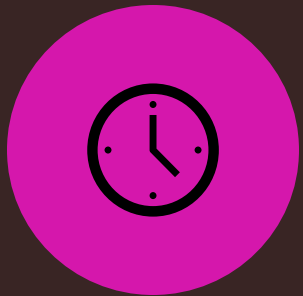
Serializabilidad o Serialización



Garantía para las transacciones, o una o más operaciones sobre uno o más objetos



Un conjunto de transacciones sobre algunos objetos se debe ejecutar como si cada transacción se ejecutara en algún pedido en serie (¡no especifica cuál!)



Sin restricciones de tiempo real, en otras palabras, no hay un orden determinista para las transacciones



"Aislamiento" en propiedades ACID

Estricta serialización

Linealización + serialización

Las transacciones tienen algún comportamiento serial y ese comportamiento corresponde al tiempo del reloj

Fácil de razonar sobre transacciones que no se superponen

¿Qué pasa con las transacciones superpuestas?

Propiedades ACID de las transacciones

Atomicidad: registros de escritura anticipada y puntos de control.

Consistencia: lógica de aplicación.

Aislamiento: controles de concurrencia (bloqueos, 2PL, OCC, MVCC).

Durabilidad: registros de escritura anticipada y puntos de control.

Controles de concurrency

Bloqueo global: simple, pero lento

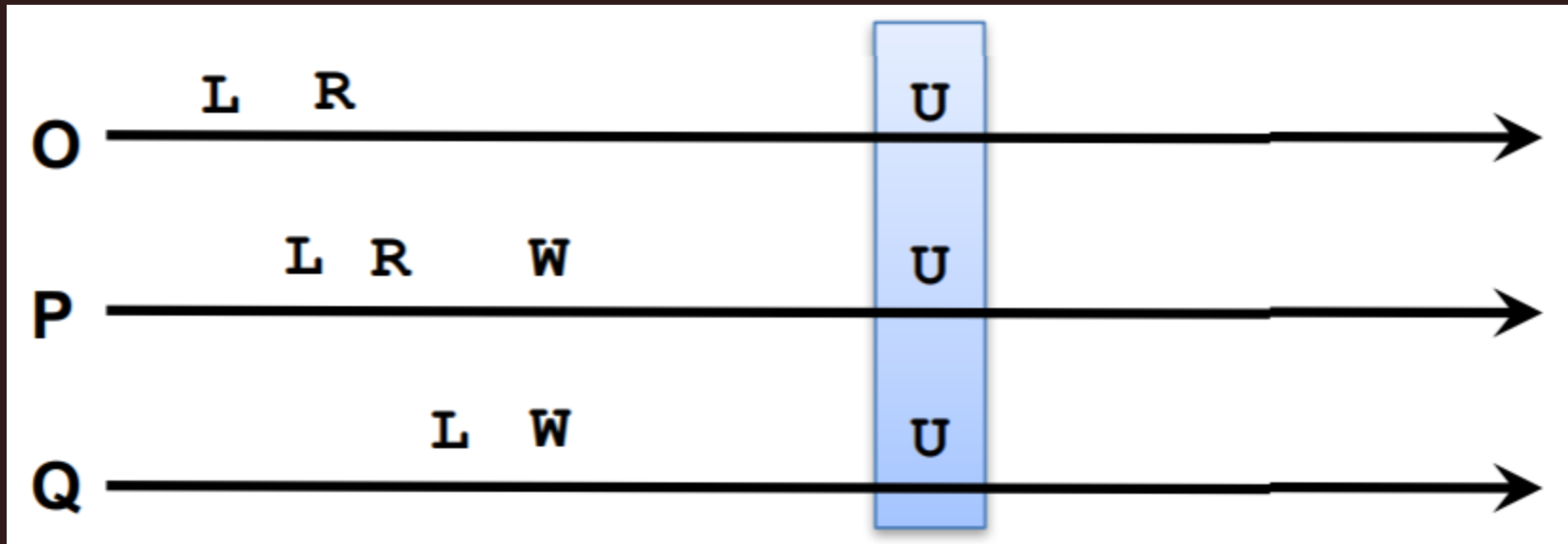
Bloqueo por objeto: no garantiza la serialización (aislamiento)

2PL: da serialización, pero deja oportunidades sobre la mesa y pueden llegar a un deadlock

OCC: funciona bien si hay pocos conflictos, pero mal si hay muchos conflictos

MVCC: aislamiento de instantáneas, no asegura serialización

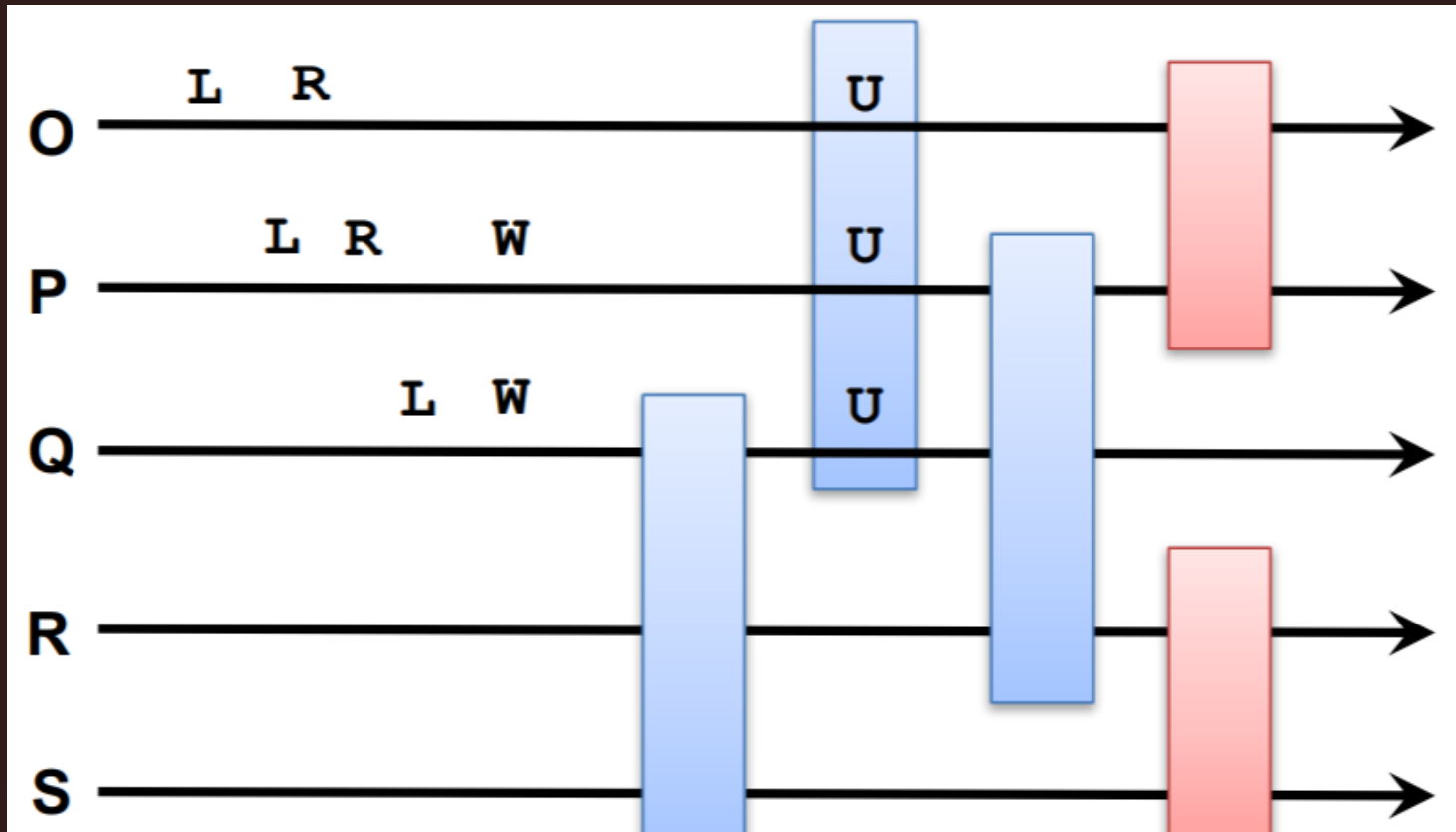
Considere datos particionados en servidores



¿Por qué no utilizar simplemente 2PL?

- Tomar los candados sobre todo el conjunto de lectura y escritura.
- Realizar escrituras
- Liberar bloqueos (en el momento de la confirmación)

Hombre de paja (Strawman): ¿Consenso por grupo txn?

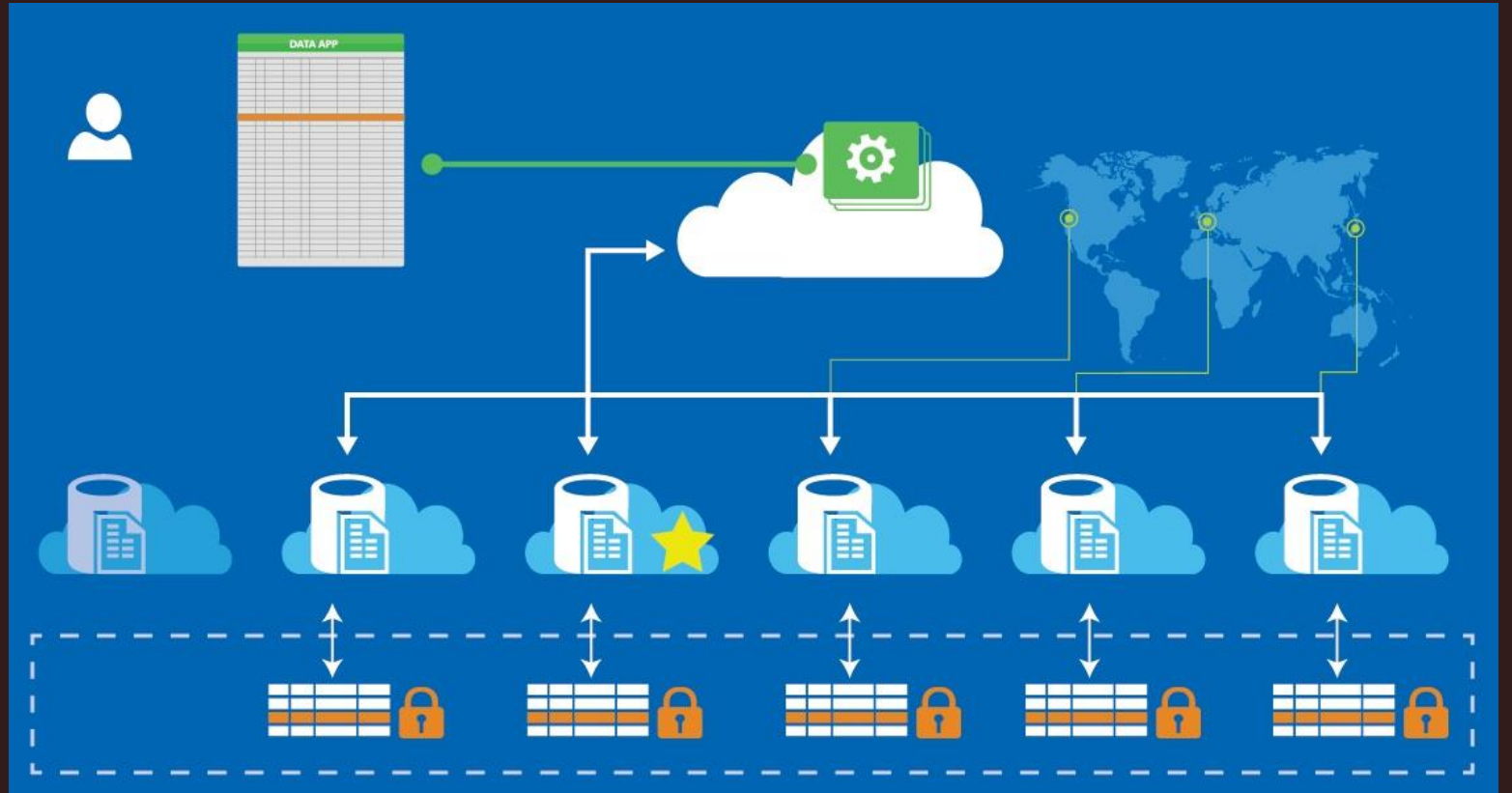


Reloj único Lamport, ¿consenso por grupo?

- ¡Forma parte de la linealización! – Pero no resuelve el problema de txn concurrentes y no superpuestas

Confluencia en Operaciones de Bases de Datos

Integrando Cambios
Concurrentes de Forma
Coherente



Confluencia en Operaciones de Bases de Datos

La confluencia en bases de datos se refiere a la capacidad de integrar cambios concurrentes realizados por múltiples nodos de manera coherente, evitando conflictos y manteniendo la consistencia en un entorno distribuido.

En entornos distribuidos, múltiples nodos acceden y modifican datos simultáneamente.

La confluencia es crucial para evitar inconsistencias y conflictos que podrían surgir de estas operaciones concurrentes.

Proporciona un marco para integrar cambios de manera ordenada y asegurar que la base de datos mantenga su integridad.

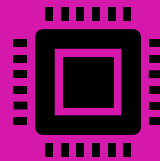
Confluencia en Operaciones de Bases de Datos



Sistemas de control de versiones: La confluencia es esencial para fusionar cambios realizados por diferentes desarrolladores en un repositorio distribuido.



Bases de datos distribuidas: Asegura que las operaciones realizadas en diferentes nodos se integren de manera coherente.



Colaboración en tiempo real: Es crucial para plataformas que admiten edición simultánea de documentos o datos.

Confluencia y coherencia



La coherencia
es esencial
para la
integridad de
los datos.

La confluencia garantiza
que la base de datos siga
siendo coherente incluso
en condiciones de alta
concurrency.

En entornos
concurrentes, la falta de
coherencia puede llevar
a resultados
impredecibles y errores.

Sloppy Quorum (Quórum descuidado)

- Un quórum se refiere a un conjunto de nodos en un sistema distribuido que deben estar de acuerdo para realizar una operación. En un entorno de bases de datos distribuidas, un quórum es crucial para garantizar la coherencia y la disponibilidad.
- La idea detrás del "Sloppy Quorum" es flexibilizar los requisitos de quórum en ciertas situaciones para mejorar la disponibilidad. En lugar de requerir la participación total de un conjunto específico de nodos, un quórum "sloppy" puede considerar un número menor de nodos para alcanzar un acuerdo. Esto se hace a expensas de una posible disminución en la coherencia de los datos.



Sloppy Quorum (Quórum descuidado)

- **Ejemplo:** Si se establece un quórum de lectura y escritura en una base de datos distribuida, un enfoque "sloppy" podría permitir que la operación continúe incluso si no todos los nodos especificados están disponibles, siempre y cuando se alcance un quórum parcial.





Muchas gracias